Junkyard Benchmarking: Assessing Repurposed Smartphone Cluster Performance in Academic Usecases

ABAS MOHAMED*, University of California, San Diego, USA
AYUSHMAAN PURI*, University of California, San Diego, USA
RICHARD VO*, University of California, San Diego, USA
SHUZHENG ZHANG*, University of California, San Diego, USA

The Junkyard Project [16] aims to repurpose old and unusable smartphones as compute units for distributed computing [15]. One target use case is academic autograding; in the most recent offering of *Intro to Parallel Computing*, 330 students shared just 40 GPU nodes on DSMLP[1], and demand routinely spiked around deadlines. Student submissions don't need raw GPU power; they need a consistent, available execution environment. Repurposed smartphones fit perfectly: power-efficient, self-contained, standardized, and sourceable at near-zero cost. A dedicated phone cluster eliminates competition for shared HPC resources. This project results in a pipeline that will estimate how many phones are needed to serve a class of n students without meaningful queuing delays.

CCS Concepts: • **Applied computing** → **Computer-assisted instruction**; *Interactive learning environments*; • **General and reference** → **Performance**; • **Networks** → *Network performance modeling*; • **Computing methodologies** → **Interactive simulation**.

ACM Reference Format:
Abas Mohamed, Ayushmaan Puri, Richard Vo, and Shuzheng Zhang. 2026. Junkyard Benchmarking: Assessing Repurposed Smartphone Cluster Performance in Academic Usecases. 1, 1 (June 2026), 21 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Project Introduction

Modern computer science education increasingly depends on shared high-performance computing (HPC) infrastructure. Universities pool GPU and CPU resources into shared clusters, giving students access to hardware they could not otherwise afford and enabling courses to assign computationally intensive work at scale. University-operated HPC systems are, however, significantly under-resourced relative to industry and national laboratories, and during peak demand researchers and students alike face long wait times that slow iteration-heavy workflows [13]. These pressures are most acute in instructional settings, where deadline-driven submission patterns cause demand to spike sharply and unpredictably.

In a recent offering of Intro to Parallel Computing at UC San Diego, 330 students shared just 40 reservable GPU nodes on the campus DSMLP cluster. During low-traffic periods the allocation was adequate. Around assignment

*All authors contributed equally to this research.
[1]UCSD's Data Science and Machine Learning Platform

Authors' Contact Information: Abas Mohamed, aam037@ucsd.edu, University of California, San Diego, La Jolla, California, USA; Ayushmaan Puri, University of California, San Diego, La Jolla, California, USA, aypuri@ucsd.edu; Richard Vo, University of California, San Diego, La Jolla, California, USA, rivo@ucsd.edu; Shuzheng Zhang, University of California, San Diego, La Jolla, California, USA, shz078@ucsd.edu.

ACM XXXX-XXXX/2026/6-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

deadlines, queues grew long enough to delay feedback by hours. This reflects a broader structural mismatch between shared, general-purpose compute and the highly variable demand patterns of large programming courses.

Automated grading systems—autograders—are now a standard tool for managing courses at this scale. They provide immediate, consistent feedback without requiring large instructional teams, and make it practical to run larger courses [6]. More frequent engagement with autograder feedback is also associated with improved student outcomes [19]. But autograders are only as effective as the infrastructure running them. If the execution environment is shared with other workloads, availability becomes inconsistent, and the feedback loop breaks down precisely when students need it most.

Critically, student submissions for an introductory parallel computing course do not require high-end GPU throughput. They need a standardized execution environment that returns consistent, reproducible results quickly. A cluster of repurposed smartphones can provide exactly this, at a fraction of the cost of purpose-built server hardware and without competing for shared HPC allocations.

The case for smartphone-based clusters is well established in prior work. Ranganathan et al. demonstrated the feasibility of running automated academic grading on a repurposed phone cluster, showing that even a four-phone cluster can process 33 grading tasks per minute [10]. Switzer et al. showed more broadly that such clusters are between 9.8× and 18.9× more carbon efficient than equivalent cloud instances [16]. Ngoy et al. further showed that e-waste smartphones can be repurposed as small-scale data centers capable of efficiently processing and storing data [7]. Beyond computational performance, the environmental case is compelling on its own: the WEEE Forum and UNITAR estimate that in 2022 alone, roughly 5.3 billion mobile phones were discarded globally, the vast majority destined for landfills or incineration despite remaining functionally intact [18]. Redirecting even a small fraction of these devices into educational infrastructure extends their useful life and offsets demand for newly manufactured hardware.

Smartphones are not without challenges as compute nodes. They are designed for battery-operated consumer use, and their aggressive power governors throttle CPU and GPU frequency under sustained load, making consistent performance harder to fully guarantee. Flashing a standard Linux distribution to replace the stock OS is a prerequisite for cluster-grade control, and the process is device-specific and labor-intensive. That said, their standardized hardware profiles make execution environments reproduible across nodes once deployed, and their ubiquity means institutions can source them at little to no cost. For the autograding use case specifically, were worloads are short-lived and latency tolerance is moderate, these tradeoffs are acceptable. A dedicated phone cluster still offers a meaningfull improvement over competing for slots on an oversubscribed shared HPC system.

This paper investigates how many phones are needed to serve a class of n students without meaningful queuing delays, and presents a principled estimation strategy grounded in real submission data from a recent course offering.

2 Related Works

2.1 Smartphone Repurposing for Distributed Computing

Early work established the basic feasibility of mobile hardware as distributed compute nodes. Büsching et al. built a six-node Android cluster running LINPACK benchmarks via MPI, showing smartphone compute capacity was comparable to workstation-class hardware of the same era [2]. Arslan et al. developed CWC, a scheduling framework that achieved 1.6× faster task completion by exploiting charging idle time [1]. Sanches proposed a mobile device cloud using a move-computation-to-data strategy to minimize inter-device data exchange [11]. These systems, however, used temporarily idle consumer devices rather than dedicated repurposed hardware. Switzer et al. addressed this directly, building a cloudlet of repurposed Pixel 3A phones and introducing the Computational Carbon Intensity metric to quantify the carbon efficiency of extending device lifetimes. Their clusters were 9.8×–18.9× more carbon efficient than equivalent AWS EC2 instances [16], a finding Switzer's

dissertation extended to a broader class of repurposed hardware [15]. Ranganathan et al. built on this platform, deploying Kubernetes and Docker on a 16-phone Pixel Fold cluster for serverless autograding and computer vision inference [10]. Ngoy et al. independently demonstrated e-waste smartphones as small-scale data centers, achievable at roughly 8 euros per device [7].

2.2 Autograding Infrastructure

Automated grading systems have become standard in large programming courses. Keuning et al. conducted a systematic literature review of 101 autograding tools, finding that most provide fully automated assessment with near-instantaneous feedback and support for multiple resubmissions [6]. Their review established that immediate feedback and consistent evaluation criteria are the primary drivers of student satisfaction and learning outcomes. Zhang et al. extended this empirical picture with an observational study across five community colleges, finding that students who check autograder feedback more frequently achieve measurably higher scores [19].

2.3 Shared HPC in Academic Settings

University-operated HPC systems are a critical resource for both research and instruction, but they remain significantly under-resourced relative to industry and national laboratory infrastructure [13]. Institutions such as Stanford, Columbia, and Georgia Tech maintain shared clusters specifically for coursework, but allocations are fixed and usage is governed by policies designed primarily around research workloads rather than the bursty, deadline-driven patterns of student submissions [13]. This mismatch is well documented: shared clusters experience long wait times and degraded throughput precisely during peak instructional demand [13]. Proposals to address this have included cloud bursting, priority-based scheduling, and dedicated instructional partitions, but each introduces either cost, complexity, or fairness tradeoffs that are difficult to resolve within a course budget.

2.4 Sustainable Computing and E-Waste

The environmental motivation for hardware repurposing is grounded in manufacturing carbon, not operational energy. Suckling and Lee's life cycle analysis of smartphones found that manufacturing dominates greenhouse gas emissions over the device lifetime, meaning that extending device life through repurposing yields a substantially better carbon outcome than recycling or disposal [14]. The scale of the problem is significant: the WEEE Forum and UNITAR estimate that 5.3 billion mobile phones were discarded in 2022 alone, forming a major fraction of the 62 million tonnes of global e-waste generated that year [18]. Despite containing valuable metals and largely functional compute hardware, the majority of these devices are hoarded, landfilled, or incinerated. Repurposing programs that redirect even a small fraction of this stock into productive use extend device lifetimes, displace new hardware manufacturing, and reduce the total embodied carbon of a computing deployment.

3 Project Overview

We propose a simulator to estimate the number of nodes needed. We are approaching the project in three general areas, as seen in Figure 1:

- Student data: Here we collect and analyze the past student data for the course offering, as well as modeling the data so it can be generated to fit a class of any scale.
- Performance metrics: Here we benchmark the real clusters and use the results as the basis for our simulator behaviors.
- Simulation: Here we generate the submissions to a class of any scale, and run the simulation to get our final results.

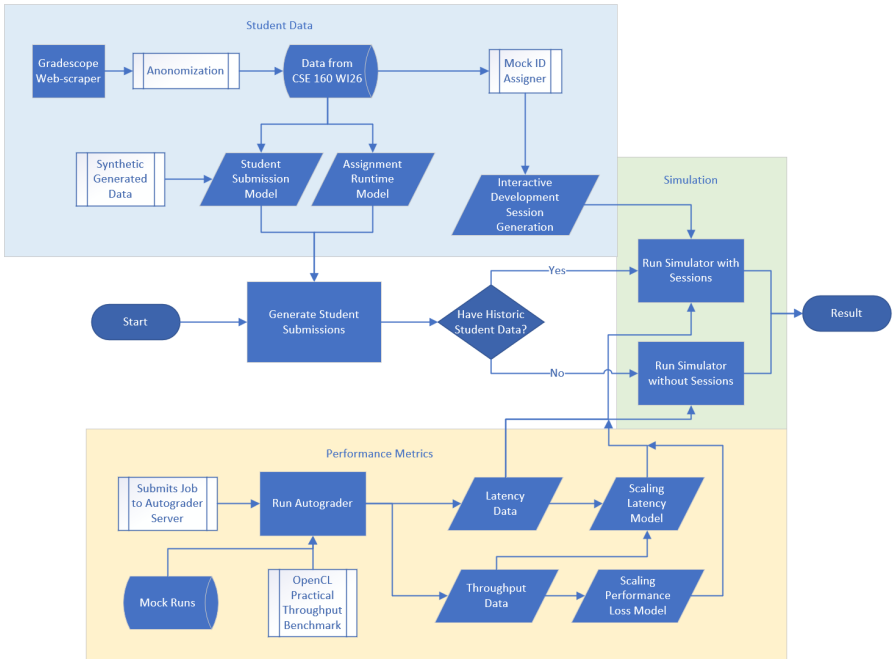


Fig. 1. Project Workflow

4 Historic Data and Analysis

4.1 CSE 160 Data

4.1.1 Data collection. Collecting data from CSE 160 - Intro to Parallel Computing at UC San Diego was constrained by two factors: Gradescope (e-grader of choice for this class) does not expose an API for bulk data export [4], and student submission records are protected under FERPA. One of the members on the team —Tom (Shuzheng)— was a part of the instructional staff for the class and therefore had to manually webscrape each programming assignment and anonymize them for the rest of the team to work with.

4.1.2 Limitations. Because data collection relied on webscraping Gradescope, we had access only to formal submission records. Interactive development sessions on DSMLP, accessed directly via SSH and independent of Gradescope, were not captured. This means that the data reflects assignment submission load but not the ambient compute demand students generate while iterating on their code. Additionally, the webscraping process aggregated results across all execution targets into a single export: submission times reflect grading runs across x86 CPU, NVIDIA GTX 1080Ti, Qualcomm Adreno 643, and Google Pixel Fold nodes without distinguishing between them.

4.2 Analysis and Modeling

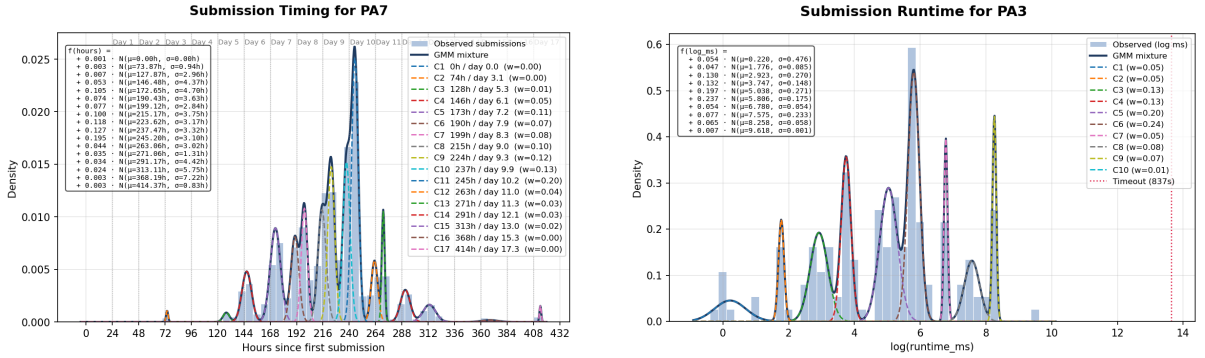
To characterize student workload patterns, we fit two independent probabilistic models per programming assignment: one capturing when submissions arrive over the assignment window, and one capturing how long submitted jobs take to execute. Both models use Gaussian Mixture Models (GMMs) selected via the Bayesian Information Criterion (BIC). All models are fit per assignment.

4.2.1 Submission Timing Model. Submission timing data was available for all programming assignments. For each assignment, submission timestamps were converted to hours elapsed since the first recorded submission, producing a non-negative scalar for each submission. This parameterization anchors all assignments to a common relative timeline regardless of absolute calendar date.

We fit a GMM to the resulting distribution of submission hours. The number of components k was selected by minimizing BIC over the range $k \in [1, \lfloor T/24 \rfloor]$, where T is the total assignment span in hours, bounding the search to at most one component per day of the assignment window. Each candidate model was initialized with ten random restarts (`n_init=10`) to reduce sensitivity to local optima, and trained for up to 500 EM iterations with full covariance. The resulting mixture takes the form

$$f(t) = \sum_{j=1}^{k} w_j \, \mathcal{N}(t \mid \mu_j, \sigma_j^2), \tag{1}$$

where t is hours since first submission, and w_j, μ_j, σ_j are the weight, mean, and standard deviation of component j, respectively, sorted chronologically by μ_j. Components correspond naturally to distinct submission bursts, which we observe to align with the days preceding the assignment deadline.



(a) GMM fitted to the submission timing for programming assignment 7

(b) GMM fitted to the runtimes of submissions for programming assignment 3

Fig. 2. Gaussian mixture model fits for PA7 timing and PA3 runtime.

4.2.2 Job Runtime Model. Runtime data was available for programming assignments 3, 4, 5, 7, and 8, which were evaluated on execution time in addition to correctness. The remaining assignments were graded for correctness only and produced no timing measurements.

Before fitting, submissions were partitioned into three categories. Pre-execution failures — submissions where the grading container failed before any student code ran — produced no runtime measurement and were excluded entirely. Timeouts, recorded as -1 in the dataset, represent submissions that either exceeded the grader's kill window or failed to secure an execution node on DSMLP during the Gradescope handoff; these were excluded from the GMM fit but their frequency was tracked separately as a timeout fraction ϕ of all executed submissions. The remaining valid runtimes formed the fitting population.

Runtime distributions in this setting are right-skewed and span several orders of magnitude, reflecting the diversity of student implementations. We therefore log-transformed valid runtimes prior to fitting, modeling $\log(\text{runtime}_{\text{ms}})$ as a Gaussian mixture, which is equivalent to modeling the original runtimes as a mixture of

log-normals. The component count k was again selected by BIC, with the search range bounded by the assignment duration in days, and each candidate model trained with ten restarts and full covariance. The mixture in log-space is

$$g(x) = \sum_{j=1}^{k} w_j N(x \mid \mu_j, \sigma_j^2), \quad x = \log(\text{runtime}_{\text{ms}}), \tag{2}$$

with components sorted by μ_j in ascending order. For each component, the median runtime in the original millisecond scale is recovered as $\exp(\mu_j)$, and the 5th and 95th percentiles as $\exp(\mu_j \mp 1.645\,\sigma_j)$.

The timeout fraction ϕ is reported alongside the GMM parameters for each assignment. Timeouts impose a distinct load pattern from ordinary jobs: rather than completing and releasing a worker, they hold a worker thread for the full duration of the grader kill window, making their contribution to cluster load disproportionate to their count.

5 Submission Generation

To simulate cluster load without requiring live student traffic, we generate synthetic submission traces by sampling from the fitted GMMs described in the previous section. Each synthetic trace represents a complete assignment submission period for a class of n students. Based on observed behavior in CSE 160, students submit approximately three attempts per assignment on average, so a class of n students produces roughly $3n$ synthetic submissions. For the reference class size of 330 students, this yields approximately 1,000 submissions per assignment.

5.1 Decoupled Timing and Runtime Sources

The generator accepts independent PA indices for submission timing and job runtime, allowing the two distributions to be drawn from different assignments. This decoupling is useful when runtime data is unavailable for a given assignment — recall that only PAs 3, 4, 5, 7, and 8 carry runtime measurements — and when the goal is to stress-test the cluster under timing patterns from one assignment with runtime characteristics from another. Unless otherwise specified, timing and runtime are drawn from the same assignment.

5.2 Sampling Procedure

Both GMMs are refit from the filtered data at generation time using the same BIC procedure described in Section 4.2. Submission arrival times are drawn by sampling N points from the timing GMM and clipping to $[0, \infty)$ to eliminate the small probability mass that full-covariance Gaussians near $t = 0$ place on negative values. Job runtimes are drawn by sampling N points from the runtime GMM in log-space and exponentiating, yielding samples from the underlying log-normal mixture:

$$\hat{r}_i = \exp(z_i), \quad z_i \sim \sum_{j=1}^{k} w_j \, \mathcal{N}(\mu_j, \sigma_j^2). \tag{3}$$

Runtime samples are rounded to the nearest millisecond. Submission scores and active-submission flags are not modeled parametrically. Instead, they are drawn from from the distributions observed in the timing PA's data.

Once all fields are sampled, submissions are sorted in ascending order of arrival time to produce a chronological trace. The output is a flat CSV with one row per synthetic submission, containing the attempt index, hours since first submission, runtime in milliseconds, score, and active flag. This trace serves as the input workload for the cluster capacity simulation described in Section 9.

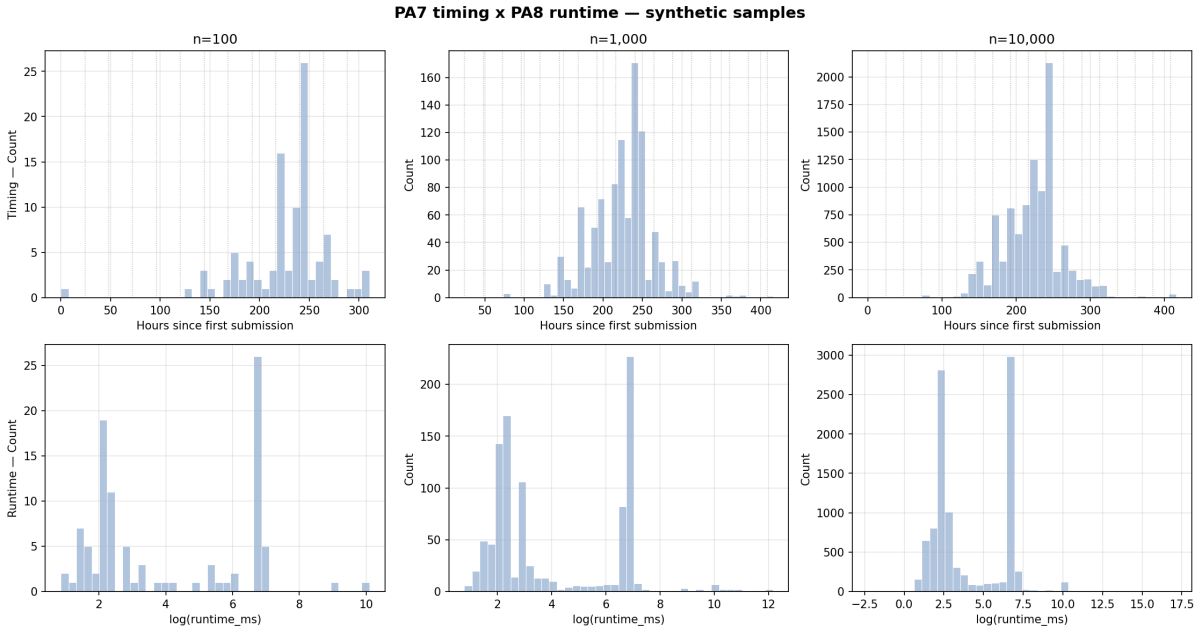


Fig. 3. Sample data generated for 100, 1000, and 10000 submissions based on PA7's submission rates and PA8's runtimes

6 Cluster Setup

We are targeting two existing experimental clusters that we have access to: the Junkyard Cluster known as Strawhat that uses repurposed Google Pixel Fold smartphones with Google Tensor G2 SoC[2], and a Qualcomm cluster that uses Thundercomm Rubik Pi 3 with Qualcomm Dragonwing QCS6490 SoC. Both clusters are running Kubernetes, and both use ARM smartphone SoCs[3], meaning that our tests can be replicated relatively easily between the clusters. Furthermore, the use of both clusters allows us to compare the performance and driver differences between the clusters.

6.1 Junkyard Strawhat Cluster

Straw-hat is an x86_64 Ubuntu 24.04 machine with two network interfaces: one facing the public internet and another one on the internal LAN that all the phones connect to. It runs k3s as the control plane for the main 40-phone cluster, with dnsmasq handling DHCP/DNS for the phones. Straw-hat acts as the jump post that you SSH into, and from there reach the phones over the internal LAN.

The phones themselves are Google Pixel Folds that are perfectly functional with some cosmetic defects that make them unsellable. These are equipped with 8 ARM Cortex CPUs, an ARM Mali G710 GPU, the Google Tensor G2 processor, 12GB of RAM, and 512GB of internal storage.

Preparing the phones requires some labor as Pixel Android (or Pixel UI) comes in the way of flashing them directly. Each phone goes through two flash stages, First, we boot it into fastboot mode, unlock the bootloader (which requires physically confirming on the phone's screen), and flash the stock Google Android factory image.

[2]System on a Chip
[3]Qualcomm Dragonwing SoC is very similar to their Snapdragon SoCs, which are used by a majority of Android smartphones on the market

After that, each phone boots into Android briefly so we can enable USB debugging through the developer options menu.

We then flash all partitions and reboot the phone into Debian. Once Debian is running, we connect the USB-C Ethernet adapter to each phone (this allows us to connect to the phone and charge it concurrently). and connect it to the network switch which connects to out jump post, straw-hat. Finally, we run our setup script to assign each phone its unique hostname and format its storage partition.

6.2 Qualcomm Rubik Pi Cluster

The Qualcomm cluster as shown in Figure 4 consists of the main tray with the power supply and the Thundercomm Ribik Pi 3, a managed switch, and a reverse proxy.

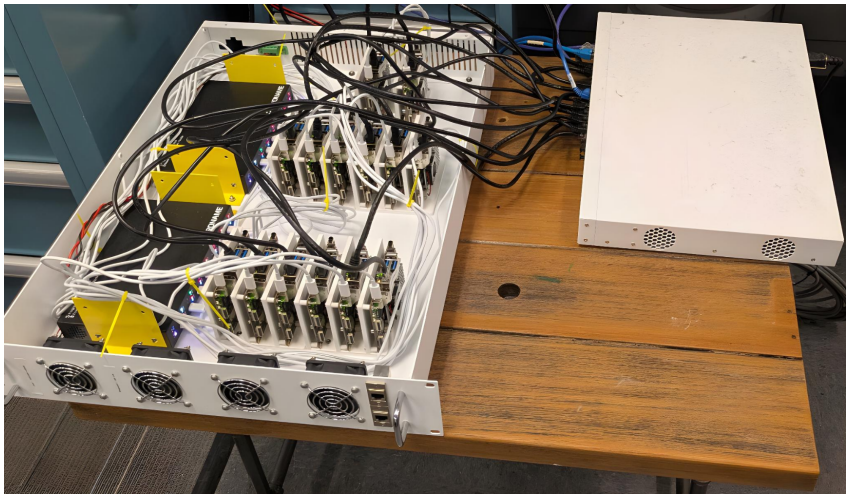


Fig. 4. The current Qualcomm Rubik Pi Cluster setup. The tray with Rubik Pi 3 picture on the left, the switch pictured on the right. Reverse proxy not in picture.

The main cluster consists of 14 Ribik Pi 3 single board computers equipped with a Qualcomm Dragonwing QCS6490 SoC each, with 8 ARM Kyro 670 CPU cores, an Adreno 630 GPU, 16 GB of onbaord LPDDR5 memory, and a Qualcomm Hexagon NPU[4], and gigabit ethernet through a dedicated 8P8C RJ45 port. One of the nodes hosts the Kubernetes Control plane, while the other 13 are worker nodes.

The Rubik Pis are supplied power through a USB PD[5] hub, and run Ubuntu 24.04 through Qualcomm's official flashing software [17]. The cluster currently runs MicroK8s, a Kubernetes distribution by Canonical, and can be access by the public internet through a reverse proxy hosted by a x86 machine that is connected to the switch. This is necessary as UCSD does not allow connections directly to the switch. The switch is managed and currently runs pfSense.

A parallel project is in progress concurrently for the Qualcomm cluster to design a custom power delivery system through the GPIO[6] headers on the boards, and will see a capacity increase as well as an upgrade in the switch used, and see the cluster fit into a standard server rack.

[4]Neural processing unit
[5]Power Delivery
[6]General Purpose IO

7 Cluster Metrics

In order to accurately simulate the cluster, we must first measure its behavior. This is important as it forms the basis of our simulator for any real cluster, and allows us to investigate any behavioral patterns specific to the cluster.

Our method comprises two key components: distributing workloads across clusters and collecting latency measurements at each stage of the auto-grading process. To replicate a real-world grading environment, we used a custom OpenCL workload designed to mirror typical classroom assignments and a scheduler previously used in CSE 160.

We executed multiple test jobs across six nodes, recording timestamps at the following stages, as seen in Figure 5 in solid blue:

(1) Job creation: A host machine from outside of the network assigns a task.
(2) Job receipt: The cluster receives the task.
(3) Cluster processing: The cluster's autograder server processes the job.
(4) Job queue: The job waits in queue until a node is available and a pod[7] is created
(5) Execution: The program runs on the assigned node.
(6) Output collection: The program returns its results.
(7) Cleanup: The pod is removed, and the node is reset for the next task.

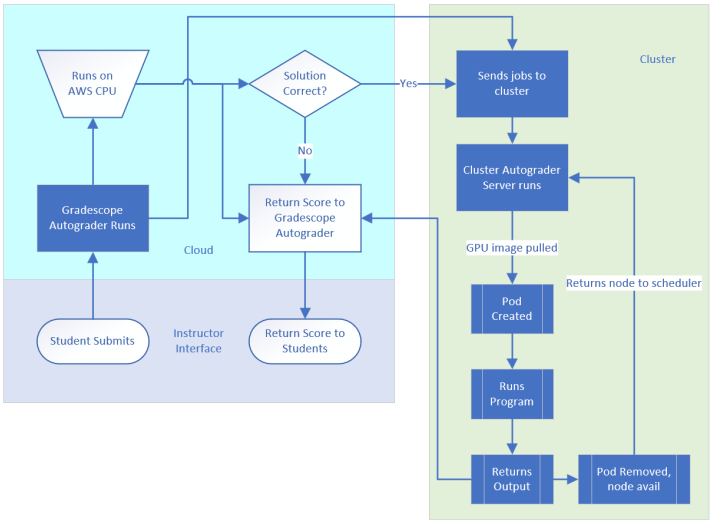


Fig. 5. Workflow for the auto-grading process in CSE 160

7.1 Mock Workload

To simulate student submissions on the cluster, we developed a mock OpenCL workload that repeatedly performs an FMA-like[8] operation for a number of iterations until a set runtime is reached. The program then calculates the total number of operations completed and returns the throughput. This allows us to measure the cluster's performance under realistic conditions.

[7] A containerized workload in Kubernetes
[8] fused multiply-add

To optimize for hardware capabilities if they are available, we also included the following flags:

- `-cl-fast-relaxed-math`: Enables aggressive mathematical optimizations.
- `-cl-mad-enable`: Activates support for MAD[9] instructions.
- `-cl-unsafe-math-optimizations`: Permits non-IEEE-compliant optimizations for faster execution.

The workload is intentionally designed to be simple and unoptimized, mirroring the structure of typical student programming assignments in CSE 160. This allows the workload to reflect real-world scenarios, where the student code is usually not very well optimized, and provides more practical metric on system performance.

7.2 Collecting Cluster Metrics

In order to collect our metrics, we first needed to complete some initial tasks: modify runner images, configure the Junkyard autograder, deploy the Junkyard autograder, create scripts to benchmark the cluster, and scripts to extract key metrics.

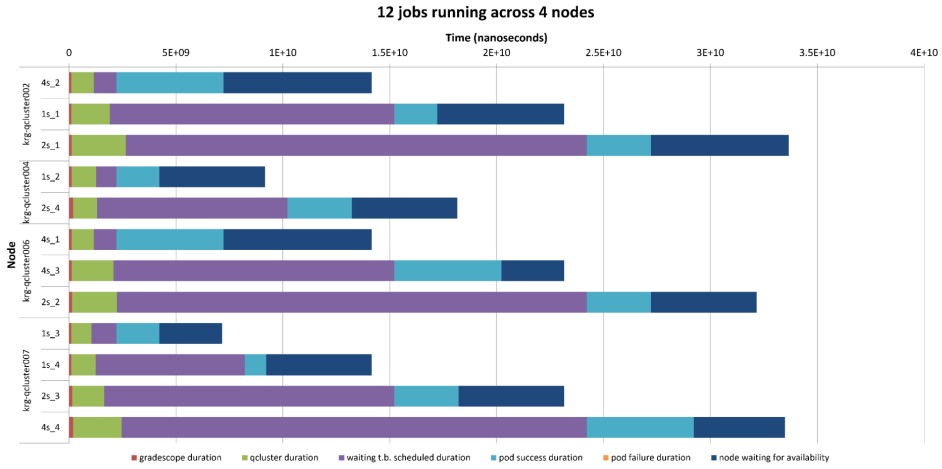


Fig. 6. Qualcomm Cluster Latencies

We ended up having to rebuild our runner images alongside configuring the Junkyard autograder for reasons related to the internals of the autograder as well as the CSE160 autograding workflow. While it is not necessary to delve too deeply into the internals of the junkyard autograder, which can be found in [8, 9], it is necessary to lay out some relevant implementation details. First, when a submission is made to the deployed Junkyard autograder server, a PA number is passed in as part of the submission. The autograder then schedules a job, where all the job does is spin up a container from our runner image, and then runs a command. Originally, this command called the PA-specific autograding script, as different assignments need to be ran and graded differently. As we only needed to run our mock OpenCL workload, we created our own simple autograding script, and changed the autograder to call that script, leading us to also have to modify our runner image to copy in that autograding script. Second, we also modified the runner images to ensure any dependencies were installed, such as `opencl-headers`, `gcc`, `make`, and etc. Third, we modified the autograder to support a variable-length set of target nodes, using Kubernetes scheduling rules. More specifically, we defined a node affinity with a node selector that matches to any `kubernetes.io/hostname`–every node has a unique hostname–inside our defined list.

[9] multiply-add

Deploying the reconfigured and rebuilt autograder was straightfoward and just involves following the step-by-step guide inside [8]. At a high level, deploying the autograder involves the following steps: building the autograder, pushing the built autograder to a GitHub Container Registry (GHCR), creating a server config yaml file, which fortunately was already made for both clusters, tweaking the configuration, and then creating a Kubernetes deployment.

With our autograder up and running, we just needed to start benchmarking and logging each job's output and the aforementioned timestamps. So, we created two main bash scripts, each with nanosecond epoch timestamps, where our main script runs locally and calls the submit script on the cluster, mirroring how "gradescope" typically submits a job to the autograder running on the cluster. We didn't have a public endpoint for the cluster autograders, so we had no choice but to benchmark in such a manner. The submit script also logs the output returned by the workload, which will be used in calculating metrics.

We also created scripts to extract the timestamps, extract the latencies, generate figures such as Figure 6, and also calculate our metrics.

Now that everything was setup, we ran benchmarks with different job counts, node counts, and workload times. Figure 6 showcases one such benchmark run, where one can see the per node jobs that ran, as well as their workload time, their iteration number, as the benchmark script looped through an array of times, and all the different timestamps recorded. We generated figures such as Figure 6 in order to sanity check the timestamps and thus latencies. However, to collect our final latency data for simulator calibration, we specifically ran a benchmark of 36 jobs of varying workload times in $1s$, $2s$, $4s$, while targeting 6 nodes. We ensured that both clusters used the same benchmark setup to uniformly collect latencies.

7.3 Performance Degradation Model

To evaluate the performance of a Kubernetes-based cluster accurately, it is essential to measure how its performance changes as the cluster scales. This involves analyzing the impact of increasing node count and job complexity on the control plane, which is a core component of Kubernetes responsible for managing cluster resources. As the number of nodes and concurrent jobs grows, the control plane may experience delays, leading to measurable performance degradation. Additionally, it is critical to confirm that the auto-grading job operates as an *embarrassingly parallel task* — meaning that the workload can be divided into independent subtasks with minimal overhead, ensuring little to no performance loss when parallelization is applied.

7.3.1 Methodology. Our approach involves running two sets of jobs with 50 2 second jobs each on the clusters using our mock OpenCL workload to quantify performance implications:

(1) Baseline Test: Jobs are executed directly on the devices to measure the native throughput. This provides a reference point for comparing the impact of overhead from the Kubernetes infrastructure. We will refer to the resulting native throughput as α for the floating point operations per second, and A for the total floating point operations.
(2) Overhead Test: Jobs are run through the Kubernetes control plane, including intermediate steps like the reverse proxy and the auto-grading server. This setup captures the additional latency introduced by the infrastructure and records timestamps for each stage of the process, and provides us with the overall throughput and performance degradation.

By comparing the results of these two tests, we isolate the effects of the Kubernetes control plane on throughput and can model the performance degradation in our simulator.

7.3.2 Results. We calculate the metrics using the following functions over n jobs on x nodes, where FLOPs[10] and FLOPS[11] are the throughput and performance for that job respectively, and Δt is the total runtime of the script:

- Throughput Excluding Latency: $\tau_{x_i} = \sum_n \frac{FLOPS}{n}$
- Throughput Including Latency: $\mathrm{T}_{x_i} = \sum_n \frac{FLOPs}{\Delta t}$
- Efficiency Excluding Latency: $\epsilon_i = \frac{\frac{\tau_{x_i}}{x}}{\alpha}$
- Efficiency Including Latency: $\mathrm{E}_i = \frac{\tau_{x_i}}{x} \div \frac{A}{\Delta t}$
- Performance Degradation: $\Delta\rho = \tau_{x_i} - \mathrm{T}_{x_i}$

Applying these functions yields the performance metric results shown in Figure 7 and Figure 8, as well as the performance degradation results shown in Figure 9 and Figure 10.

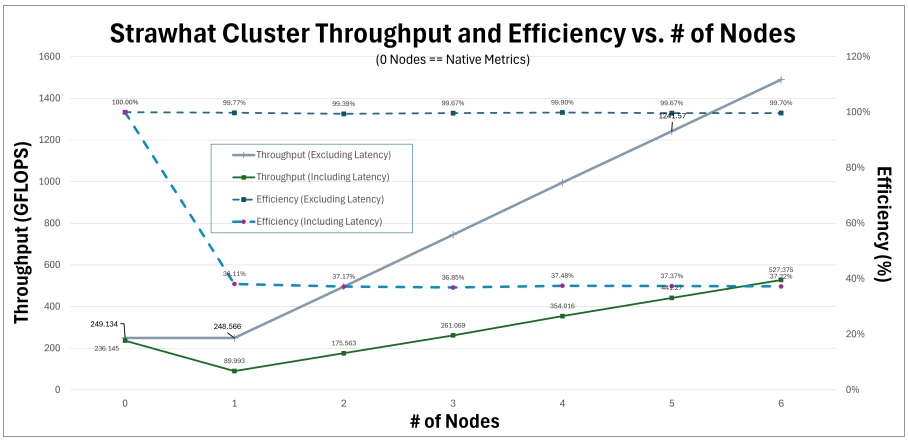


Fig. 7. Junkyard Cluster Metrics

Figures 7 and 8 reveal two important trends when validating scability in the clusters. First, the efficiency excluding latency is fairly constant at 100% across both clusters, with a slight deviation due to some random experimental errors. In other words, each node itself suffers negligible performance loss while running the workload in comparison to our baseline. This trend is to be expected, but also necessary, as the only difference between the baseline test and the overhead test, in terms of how the workload is run, is that the overhead test runs the workload inside of a runner docker/container image. Second, and most notably, efficiency including latency sees a large drop across both clusters, even starting from 1 node, but also stays relatively constant as we further scale up, meaning that the latency overhead is a fixed cost that does not scale with the number of nodes. This is key in evaluating scalibility, as we did not wish to see multiple unpredictable drops as we continued scaling up the number of nodes.

Additionally, Figures 7 and 8 reveal interesting differences in the overall performance of the clusters, albeit not related to their scalability factor. It is apparent that the Strawhat cluster has a higher native throughput, which explains the higher throughput when comparing an identical number of nodes between the clusters. However, the actual percentage drop in efficiency due to the latency overhead is around 13% more for the Qualcomm cluster as compared to the Strawhat cluster. One possible contribution to this discrepancy is the reverse proxy that is used for the Qualcomm cluster could be adding on more overhead as compared to the Strawhat cluster. Furthermore,

[10]Cumulative Floating-Point Operations
[11]Floating-Point Operations Per Second

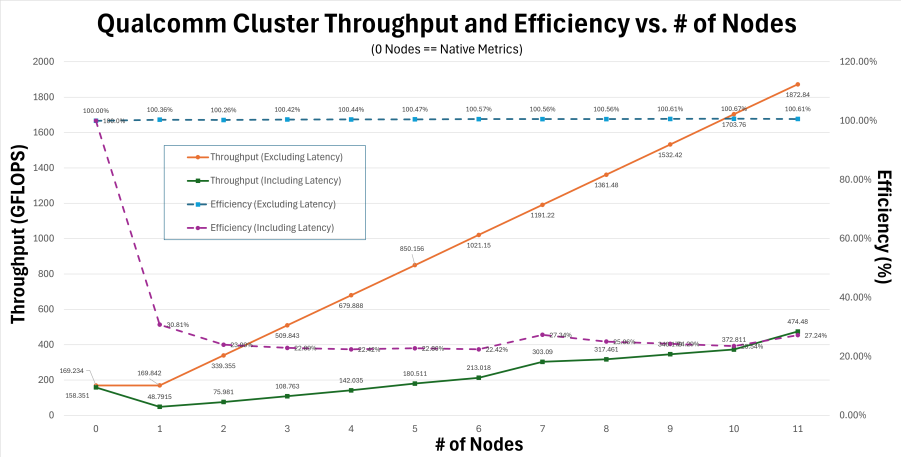


Fig. 8. Qualcomm Cluster Metrics

the quality of the network interface controller (NIC) for the Qualcomm cluster could be another contributing factor to a lower effiency including latency.

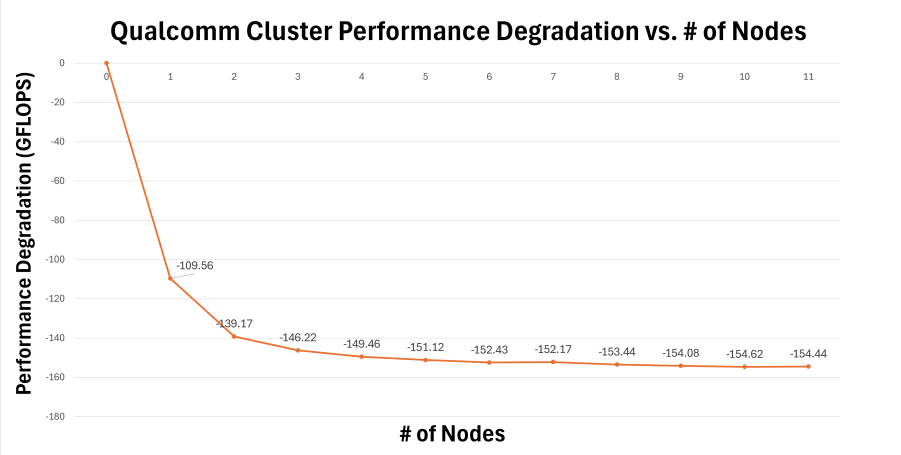


Fig. 9. Qualcomm Cluster Performance Degradation

We can see from Figure 9 and Figure 10 that the performance degradation due to latency scales in an exponential decay model that converges, we can model this with:

$$\gamma \approx -155(1 - e^{-1.2x})$$

for the qualcomm cluster, and:

$$\gamma \approx -219(1 - e^{-1.08x})$$

for the strawhat cluster.

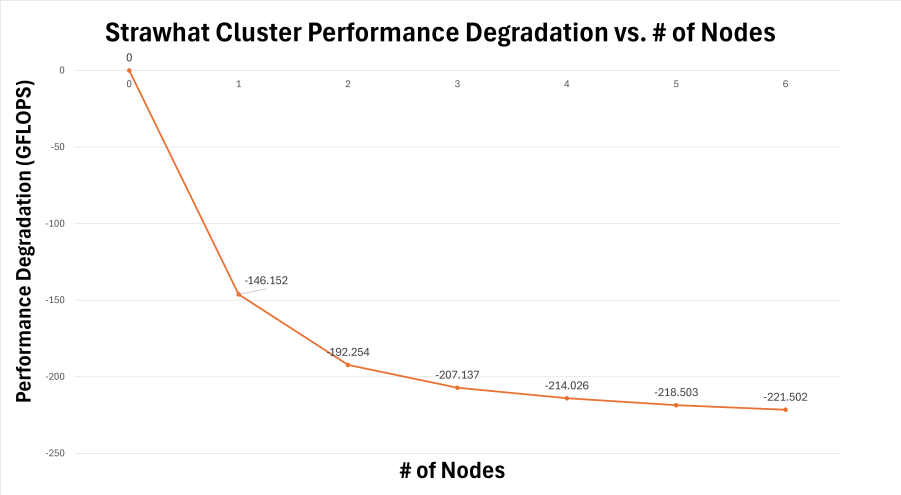


Fig. 10. Strawhat Cluster Performance Degradation

Because we are already including the latency results from running the auto-grader on 6 nodes in our simulator, and that $\Delta\gamma$, the difference in degradation, is minuscule and within the margin of error, we can safely assume that we do not require a degradation model as the number of nodes increase, and that implementing the numerical overheads alone with suffice.

Note that we did not have access to more than 6 nodes of devices for the junkyard cluster, therefore we can only assume that the performance degradation as seem in Figure 10 will exhibit similar behavior to the qualcomm cluster.

7.3.3 Limitations. Because the control plane is hosted on a Google Pixel Fold for the Junkyard cluster and a Rubik Pi for the Qualcomm cluster, the network throughput from the control plane to the kubelets[12] is limited, and we expect that this connection will saturate as the number of nodes continue to increase. Once it does, we expect to see a sharp decline in the cluster efficiency including overhead, and a dramatic increase in performance degradation. However, we are unable to test this as we are limited by the number of devices we have access to.

8 Student Interactive Development Sessions

In a modern university computer science course, a cluster has two primary functions: to support students during development, and to carry out the auto-grading of assignment submissions. We have covered the auto-grading function of the cluster and will now consider its uses when supporting student development.

8.1 Background

Interactive student development sessions allocate a virtual machine on a cluster node so students can write and test code in the exact environment where it will be graded. For a course like CSE 160, where grades depend on runtime after optimization, testing on the precise GPU model is the only reliable way to meet assignment requirements — driver differences between machines are not cosmetic. CSE 160 attempted to mitigate DSMLP pressure by introducing development containers that students could run locally for correctness checks before submitting for timing. This introduced its own problems: Apple deprecated OpenCL in 2018, forcing course staff

[12]Kubernetes cluster worker nodes

to use PoCL[13] on macOS, which has implementation differences that cause code to pass locally but fail in the autograder. The feedback loop collapse is also self-reinforcing — students whose submissions time out resubmit repeatedly while holding their allocated GPU as long as possible, which increases queue pressure for everyone else. A dedicated cluster sized correctly for the course would break this cycle. Generating that size estimate requires session data, which DSMLP does not expose; the following section describes how we derive it from submission records instead.

8.2 Approach

We observe that humans have the tendency to complete tasks in bursts, which in this course specifically describes that students tend to work on nothing but their homework before making a submission and letting the auto-grader run. Once they have the results, either they continue working to improve upon it or leave it to come back another time. This is important as it gives us a way to extract development activities from the students from submissions alone.

As an example, Figure 11 is a student's submission data on PA[14] 8 in CSE 160. We can see that this student worked on the assignment on March 10th and made two submissions, before pausing and resuming the next day as there is a clear gap between the submissions. The student's program run-time and score also generally improves as they spend more time on this assignment.

attempt	submission_time	run_time_ms	score
1	2026-03-10T21:17:37.636715-07:00	-1	64.286
2	2026-03-10T22:42:06.798529-07:00	-1	0
3	2026-03-11T07:47:24.553272-07:00	17.066	90
4	2026-03-11T08:13:48.915845-07:00	30.429	90
5	2026-03-11T08:26:52.376290-07:00	17.652	90
6	2026-03-11T08:41:15.973526-07:00	9.042	90
7	2026-03-11T09:04:13.785679-07:00	-1	64.286
8	2026-03-11T09:08:08.300582-07:00	5.174	90
9	2026-03-11T09:26:35.154799-07:00	4.492	90
10	2026-03-11T09:44:46.911062-07:00	5.322	90
11	2026-03-11T09:45:18.886845-07:00	5.357	90
12	2026-03-11T10:05:00.250051-07:00	4.536	90
13	2026-03-11T10:33:08.673065-07:00	4.154	90
14	2026-03-11T10:59:26.141049-07:00	4.142	90

Fig. 11. A list of one student's submissions for PA8, CSE 160, Winter 2026

As our Gradescope-scraper retrieves the submission data of a student and logs them together, the output will be in order of their attempt, and so we can then safely assign a mock student ID to the student, forming the basis of our analysis as we now have all submissions made by one student.

This type of event is described in *Measuring and modeling bursty human phenomena* [5], which we will be using as the basis for development sessions. A few approaches to detect a burst were described, however, we will simply detect if a submission is inside of a burst by seeing if τ, the time between submissions, is greater than 3 hours, as we have determined it is very unlikely that a student will be continuously working on an assignment if they make no submissions within 3 hours.

As shown in Figure 12, the submissions are depicted as vertical lines on the time axis, and a burst of submissions depicted as the blue dotted box of time Δt is then assigned to them. We will require the following parameters for our generator, which can be changed from the default by the instructor for the specific course and assignment:

[13]Portable Computing Language
[14]Programming Assignment

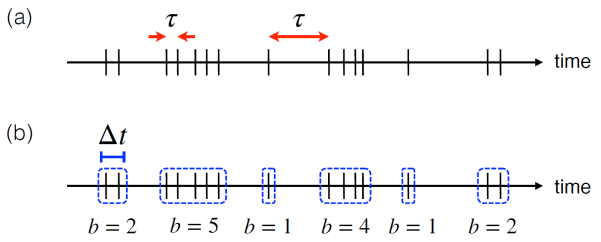


Fig. 12. "Schematic diagrams for bursty time series analysis" by Karsai and Jo, used under CC BY 4.0. Unmodified. [5]

- $t_{pre} \in \mathbb{Q}^+$: The expected development time before the first submission, modeled with the default Gaussian distribution of $\bar{x} = 3$ hours, $s = 1.0$
- $\tau_{max} \in \mathbb{Q}^+$: The maximum interval between submissions (τ) to be considered as within a burst
- $l \in \mathbb{Q}^+$: Randomized lingering time after last submission, modeled with the default Gaussian distribution of $\bar{x} = 0$ hour, $s = 0.1$

We then go through the student's submissions and group them into bursts by τ_{max}, and we now have the discrete array of burst length $\Delta t[n]$ and session start time $x[n]$. We also have the normally generated $t_{pre}[n]$ and $l[n]$ per session, and after combining the parameters, we get the final development sessions for that student:

$$sessions_a = \sum_{i=1}^{n} x[i] - t_{pre}[i], lengths_a[i] = \Delta t[i] + l[i]$$

Where a is the specific student, n is the number of sessions, and i is one specific session.

8.3 Limitations

However, this strategy is severely limited, as we require historic student data as the basis for session generation, making synthetic data incompatible. Furthermore, because we require submissions from individual students, most students do not make enough submissions so that they are statistically significant, and therefore we cannot model and scale the interactive development sessions. It is also not applicable to assignments where the submissions have a rate limit, as it artificially influences the data. Therefore, we can only use this data in our simulator for a class size of the exact number of students that were in CSE 160.

9 Simulator

Through the collection of historical student data and creating the gaussian mixture model. To the gathering of cluster metrics; we have the key components to build our simulator. But the simulator itself can have drawbacks depending on the implementation. What we are proposing in this section is

- Student submission only simulator is accurate but not realistic in node count estimation for a classroom.
- Student submission and interactive development session simulator is directly applicable to classroom node count but its backing data has clear limitations as seen previously.

9.0.1 Similarities between the two simulators. These two simulators, although encompassing different scopes of classroom node estimation, still share key points. One of these points is how we get the gathered metrics to be inserted into the simulator. In both simulators, there are operations like de-queuing a job off the main queue which has some delay in the real cluster. It would be inefficient to hard-code different cluster metrics in the simulator. Therefore, we take the approach of making a JSON key value pair for each cluster where the simulator

will dynamically read in the latencies so it can be applied at runtime as seen in figure 13. Additionally, the simulators also share the pattern of reading in a CSV of the submissions(and sessions for the student submission and interactive development session simulator) which will have the same formats in both.


```
{
    "qcluster": {
        "gradescope_overhead": 183.47361685,
        "cluster_duration_overhead": 10779.870,
        "scheduler_wait_overhead": 24382.86116,
        "pod_creation_overhead": 3671.389830
    }
}
```


Fig. 13. QCluster[15] metrics that will be injected into the simulator

9.1 Student Submission only Simulator

Student submissions are handled by the Gradescope platform[3] that will be routed and sent to the cluster as a submission batch job. The cluster will then run the students program and return the student grade back to Gradescope.

9.1.1 Implementation. There is nothing differentiating the batch jobs(student submissions) submitted to the cluster so a single queue(FCFS) for handling job to node assignment will suffice. To prevent submission jobs from waiting in the queue for too long and to match what would occur in a classroom we have a parameter called *TIMEOUT_MS*. Which is how long a submission will wait in the queue before being dropped. And finally where the simulator does the bulk of the work that can only be done in a simulated environment is find the ideal number of nodes needed to support a class with a drop rate under a threshold. As seen in Figure 14, the drop rate target; what we would want to see in a class was set to <= 5% or 5 percent or less of all jobs can be dropped. But

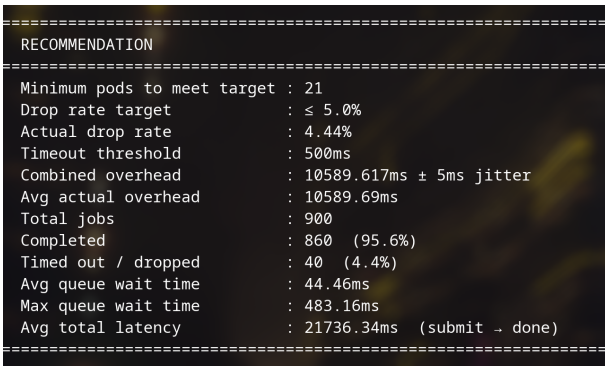


Fig. 14. Student submission simulator results

a drawback that can be seen from Figure 14 is that the timeout is very low 500*ms* or half a second. In a regular classroom setting it would be more realistic to have a submission job timeout in a range around 30 seconds to 1 minute. The reasoning that caused this to happen is that there is little concurrency going on, which results in most submission jobs getting a node quickly. In order to mitigate this and scale it closer to what would appear in the classroom we needed to apply a stricter timeout which yielded a more reasonable node count. Additionally given the environment of CSE 160, students need to run and develop their programs on these virtual machines

that have unique drivers. Without development on these virtual machines, errors unrelated to correctness but rather porting would arise. This results in a simulator that only accounts for submission jobs not representing a classroom's behavior accurately where student develop on clusters and submit to the cluster.

Finally, acknowledging these constraints, we can accurately estimate the node count for a given classroom that only has submission jobs. And even more specifically estimate the node count needed for different assignments as seen in figure 15. For two different workload assignments, we can directly look up the nodes needed to support x number of students.

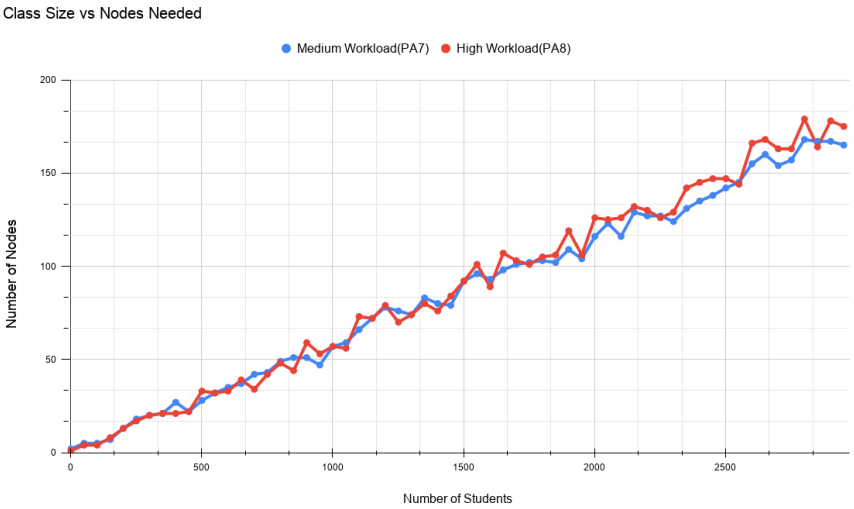


Fig. 15. Class Size vs Nodes Needed for two different workloads

9.2 Student Submission and Interactive Development Session Simulator

This simulator fixes the weaknesses of the previous simulator by giving more realistic numerical values for the parameters seen in the previous simulator. Such as *TIMEOUT_MS* changing from half a second to 300,000 milliseconds(5 minutes) as seen in figure 16. That is a much more reasonable number in a classroom. Additionally, it accounts for students developing on the nodes using the session generation method described in section 5.

9.2.1 Implementation. When two students, one who wants to get a node for development and one who is trying to submit an assignment to Gradescope, who should get precedence for a node? We answered this from this perspective. Students who lose critical development time cannot get this lost time back. Meaning they may not be able to get the chance to submit. Additionally, submissions would be less attempted since the student who is developing is given adequate chance to develop. And finally submissions can be reattempted.

To model this behavior, we use a multi-level queue [12] where interactive sessions are given the highest priority and submissions jobs are placed in a lower priority queue. This also has the side affect that interactive development sessions will never be dropped. Students who go over the session limit(6 hours in CSE 160) will be re-queued in the sessions queue and they will not timeout and can wait indefinitely. Since we now have an additional factor to think about in our simulator we must add additional parameters to model this change. So we add two new parameters for the sessions, *SESSION_WAIT_TARGET_MS* which is the amount of time we

would want to wait in the queue. Or how long a student may wait to get a node to develop on. And since it is not realistic for all students to hit this target we aim to have x percent of the sessions wait $\leq$ our parameter *SESSION_WAIT_PERCENTILE*. This can be seen again in our sim from Figure 16 where we would want 97 percent of all students to wait under a certain time to get a node to develop on. Finally, our simulator will sweep different numbers of nodes and find the minimum number of nodes that fulfill our parameters.

```
======================================================================
  RECOMMENDATION
======================================================================
  Minimum pods to meet both targets : 56

  ── Batch jobs ──────────────────────────────────────────
  Drop rate target    : ≤ 5.0%
  Actual drop rate    : 1.82%
  Timeout threshold   : 300000ms
  Overhead source     : cluster_metrics.json → 'qcluster'
  Combined overhead   : 39017.59ms (183.47361685ms gs + 10779.87ms cluster +
  Avg actual overhead : 39017.67ms
  Total / done / drop : 1101 / 1081 / 20
  Avg queue wait      : 1527.07ms
  Max queue wait      : 238146.75ms
  Avg total latency   : 96061.07ms  (submit → done)

  ── Interactive sessions ────────────────────────────────
  Session cap         : 6h  (remainder re-queued)
  Sessions never dropped (wait indefinitely)
  p97 wait target     : ≤ 100s
  Actual p97 wait     : 0.00s
  Total / completed   : 481 / 481
  Total slices run    : 521  (40 continuation slices)
  Avg initial wait    : 9294.77ms
  Max initial wait    : 1043184.84ms
  Wait percentiles    : p50=0.0s  p75=0.0s  p90=0.0s  p95=0.0s  p99=455.3s
======================================================================
```

Fig. 16. Node count for classroom with student development sessions and gradescope submissions

10 Milestones

- **M0: Project specifications**
  Lead: All *Progress: Completed*
  The document that details project goals and specifics.

- **M1: Historic Data Collection**
  Lead: Tom (FERPA Access required) *Progress: Completed*
  Historic data from CSE 160 as a baseline for simulations. Rest of the group was blocked till Tom finished scraping and anonymizing the data.

- **M2: Student Submission Model**
  Lead: Ayushmaan and Richard *Progress: Completed*
  Gaussian Mixture Model based on student submission times and code runtimes.

- **M2.5: Synthetic OpenCL Workload**
  Lead: Tom *Progress: Completed*
  This milestone was added because we realized we need some sort of program to realistically stress the pixel folds, and also we want to get the throughput data

- **M3: Virtual Simulator**
  Lead: Abas *Progress: Completed*
  A discrete-event simulator that sweeps pod counts (1 -> N) to find the minimum number of compute pods needed to keep student autograder job timeouts at or below a target drop rate.

- **M4: Mock Submission Generator**
  Lead: Ayushmaan *Progress: Completed*
  Samples submission times and runtimes to generate a csv file with a desired number of mock submissions. *Milestone modified from Scheduler Integration and Virtual Testing, as original plan was actually in scope with Milestone 3 and merged into it.*

- **M4.5: Mali Container**
  Lead: Tom *Progress: Completed*
  Build container capable of compiling and running OpenCL programs on the Pixel Folds.

- **M5: Physical Cluster Validation**
  Lead: Ayushmaan, Richard, and Tom *Progress: Completed*
  Get latencies and throughput of the three available clusters: Strawhat, Qualcomm, and Junkyard. Then, validate results of our simulator by running real-time workloads on the number of nodes given by the simulator. We had to wait for the another group's progress to be able to use qcluster.

- **M6: Mock interactive development node usage**
  Lead: Tom for model, Abas for simulator modifications *Progress: Completed*
  This milestone was added since we would need some way to model and simulate student interactive development sessions on the nodes based on student submissions, and then generate a list for the simulator.
- **M7: Final paper draft**
  Lead: All *Progress: Completed*
- **M8: Final paper and video**
  Lead: All *Progress: Completed*

11 Conclusion

Re-purposing phones that offer classes compute for student development and auto-grading on GPUs is a task that has clear benefits. However, if students cannot rely on these devices to effectively do their classwork. Then many of the same issues that some students have experienced on other kinds of distributed compute begin to appear again. Junkyard Benchmarking addresses this point by contributing a method to effectively measure the amount of compute needed to support a class. Such that issues like dropped jobs, or a lack of nodes for development can be mitigated. In addition, provide courses with a method to know how much compute they need before even beginning instruction. And this is through our simulator that relies on the GMM and cluster metrics to model the class without having the class present. In future iterations of this project, we may need to collect data on sessions rather than using mock data. And this would allow us to model student behavior more accurately.

Through benchmarking, we can get a solid idea of how the cluster behaves, which in return tells us what it does well and where it can improve. Junkyard Benchmarking bridges this gap between those who will use the clusters for their everyday tasks such as in CSE 160 and those who develop the cluster. Allowing everyone to yield the full benefits of the cluster.

References

[1] M. Y. Arslan, I. Singh, S. Singh, H. V. Madhyastha, K. Sundaresan, and S. V. Krishnamurthy. 2015. CWC: A Distributed Computing Infrastructure Using Smartphones. *IEEE Transactions on Mobile Computing* 14, 8 (Aug. 2015), 1587–1600. doi:10.1109/TMC.2014.2362753

[2] Felix Büsching, Sebastian Schildt, and Lars Wolf. 2012. DroidCluster: Towards Smartphone Cluster Computing – The Streets are Paved with Potential Computer Clusters. In *2012 32nd International Conference on Distributed Computing Systems Workshops*. IEEE, 114–117. doi:10.1109/ICDCSW.2012.59

[3] Gradescope. [n. d.]. Gradescope Autograder Documentation. https://gradescope-autograders.readthedocs.io/. Accessed: 2026-06-12.

[4] Gradescope. 2026. Gradescope Guides. https://guides.gradescope.com/hc/en-us.

[5] Márton Karsai and Hang-Hyun Jo. 2024. Measuring and Modeling Bursty Human Phenomena. arXiv:2412.13617 [physics.soc-ph] https://arxiv.org/abs/2412.13617

[6] Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A Systematic Literature Review of Automated Feedback Generation for Programming Exercises. *ACM Transactions on Computing Education* 19, 1, Article 3 (Sept. 2018), 43 pages. doi:10.1145/3231711

[7] Perseverance Ngoy, Farooq Dar, Mohan Liyanage, Zhigang Yin, Ulrich Norbisrath, Agustin Zuniga, Joao Pestana, Marko Radeta, Petteri Nurmi, and Huber Flores. 2025. Supporting Sustainable Computing by Repurposing E-Waste Smartphones as Tiny Data Centers. *IEEE Pervasive Computing* (2025). doi:10.1109/MPRV.2025.3541558

[8] Ajay Ramesh Ranganathan, Christopher L. Crutchfield, Arunan Thiviyanathan, Tom Zhang, and Nathan Liu. 2026. Junkyard Autograder. https://github.com/junkyard-computing/junkyard-autograder. GitHub repository, accessed June 12, 2026.

[9] Ajay Ramesh Ranganathan, Risab Sankar, Grant Cheng, Ayan Gaur, Arunan Thiviyanathan, and Jeffrey Lee. 2025. Junkyard Computing: Repurposing Discarded Smartphones for Serverless Applications. https://kastner.ucsd.edu/wp-content/uploads/2025/06/admin/junkyard.pdf Accessed: 2026-06-12.

[10] Ajay Ramesh Ranganathan, Risab Sankar, Grant Cheng, Ayan Gaur, Arunan Thiviyanathan, and Jeffrey Lee. 2025. Junkyard Computing: Repurposing Discarded Smartphones for Serverless Applications. (2025), 16 pages. https://kastner.ucsd.edu/wp-content/uploads/2025/06/admin/junkyard.pdf

[11] Pedro Miguel Costa Sanches. 2017. *Distributed Computing in a Cloud of Mobile Phones*. Ph. D. Dissertation. Universidade NOVA de Lisboa.

[12] Iqra Sattar, Muhammad Shahid, and Nida Yasir. 2014. Multi-level queue with priority and time sharing for real time scheduling. *International journal of multidisciplinary sciences and engineering* 5, 8 (2014), 16–17.

[13] Peng Shu, Junhao Chen, Zhengliang Liu, Huaqin Zhao, Xinliang Li, and Tianming Liu. 2025. Survey of HPC in US Research Institutions. *arXiv preprint arXiv:2506.19019* (2025). https://arxiv.org/abs/2506.19019

[14] Josh Suckling and Jacquetta Lee. 2015. Redefining Scope: The True Environmental Impact of Smartphones? *The International Journal of Life Cycle Assessment* 20 (2015), 1181–1196. doi:10.1007/s11367-015-0909-4

[15] Jennifer Switzer. 2025. *Hardware Repurposing to Reduce the Embodied Carbon of Computing*. Ph. D. Dissertation. University of California, San Diego. https://escholarship.org/uc/item/6g54p6pk UC San Diego Electronic Theses and Dissertations. ProQuest ID: 32400617.

[16] Jennifer Switzer, Gabriel Marcano, Ryan Kastner, and Pat Pannuto. 2023. Junkyard Computing: Repurposing Discarded Smartphones to Minimize Carbon. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 400–412. doi:10.1145/3575693.3575710

[17] Thundercomm. 2026. RUBIK Pi 3 Documentation. https://www.thundercomm.com/rubik-pi-3/en/docs/rubik-pi-3/en/docs/1.0.0-u.

[18] WEEE Forum and UNITAR. 2022. *Of 16 Billion Mobile Phones Possessed Worldwide, 5.3 Billion Will Become Waste in 2022*. Technical Report. Waste Electrical and Electronic Equipment Forum / United Nations Institute for Training and Research. https://unitar.org/about/news-stories/news/16-billion-mobile-phones-possessed-worldwide-53-billion-will-become-waste-2022

[19] Adam Zhang, Heather Burte, Jaromir Savelka, Christopher Bogart, and Majd Sakr. 2025. Auto-grader Feedback Utilization and Its Impacts: An Observational Study Across Five Community Colleges. *arXiv preprint arXiv:2507.14235* (2025). https://arxiv.org/abs/2507.14235